



Ciência da Computação
Engenharia de Software

Ana Julia da Silveira
Arthur Alves Vargas
Arthur Gabriel da Silva
Bernardo Ruís Karnopp Bessow
Floriano Pellegrini Laste
Wellington Dias Bergmann

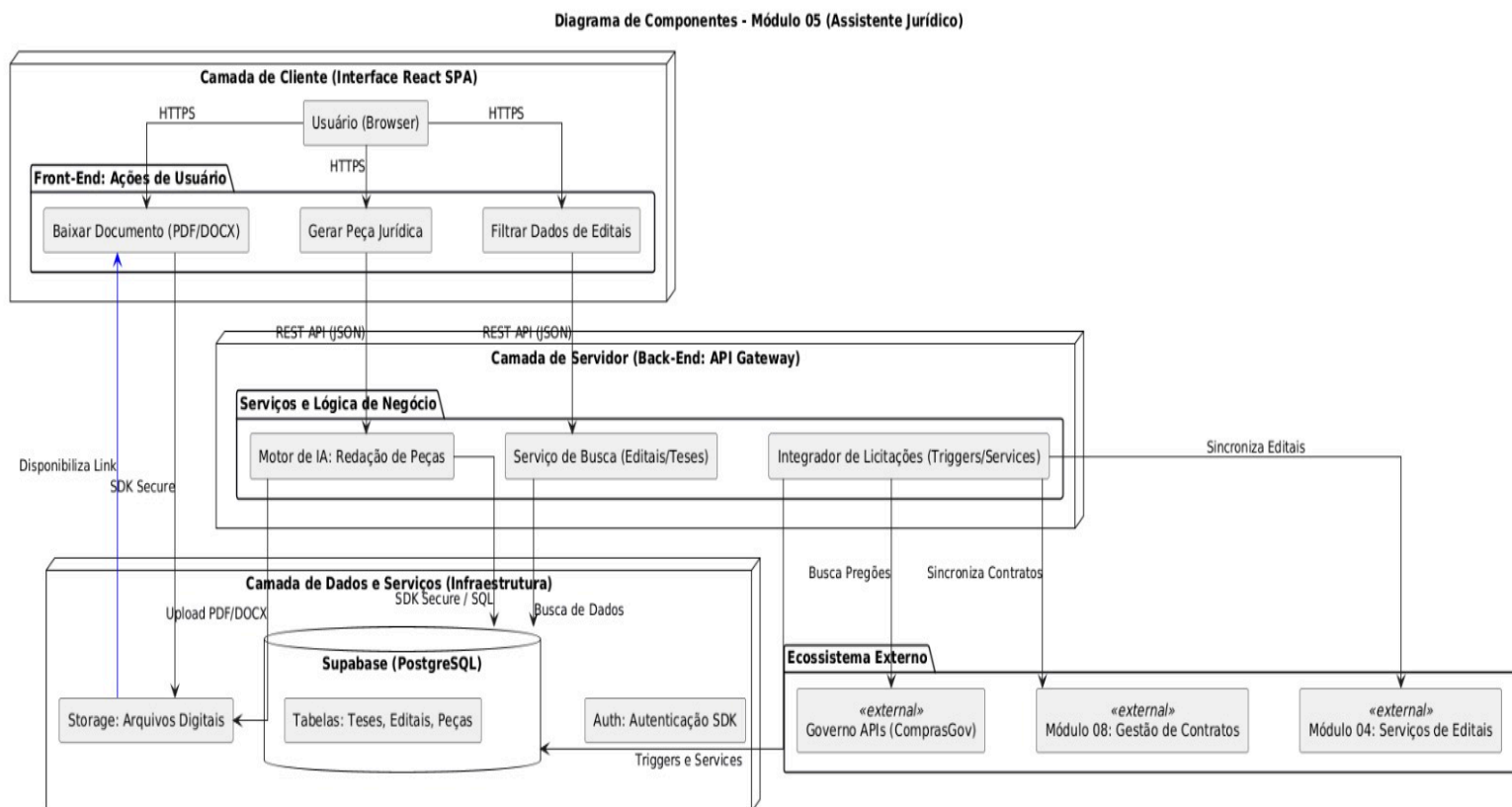
DOCUMENTO DE ARQUITETURA E DIAGRAMAS UML

MÓDULO 05 - ASSISTENTE JURÍDICO

Santa Cruz do Sul

2026

1. Diagrama de Componentes



Detalhamento:

→ Camada de Cliente (Interface React SPA): O usuário interage com uma aplicação do tipo Single Page Application (SPA), na qual o navegador carrega a estrutura principal da página em uma única vez. A partir disso, as principais funcionalidades são expostas como ações diretas:

- **Filtrar Dados de Editais:** Permite a consulta refinada na base de dados para selecionar o alvo desejado.
- **Gerar Peça Jurídica:** Dispara o fluxo principal de automação para o back-end, abrindo um pop up que permite de forma guiada realizar a criação do documento jurídico.
- **Baixar Documento (PDF/DOCX):** Realiza o download seguro do arquivo final, que será armazenado no histórico de peças jurídicas.

→ Camada de Servidor (Back-End: API Gateway):

- Esta camada atua como o cérebro do sistema, processando as requisições via REST API.
- Motor de IA (Geração de Peças): O componente central que utiliza inteligência artificial para sugerir temas e argumentos concretos, baseados em precedentes de licitações anteriores para geração de peças jurídicas.
- Integrador de Licitações (Triggers/Services): Serão responsáveis pela sincronização contínua com outros módulos, garantindo que o sistema sempre tenha dados atualizados sem intervenção manual.

→ Camada de Dados e Infraestrutura (Supabase):

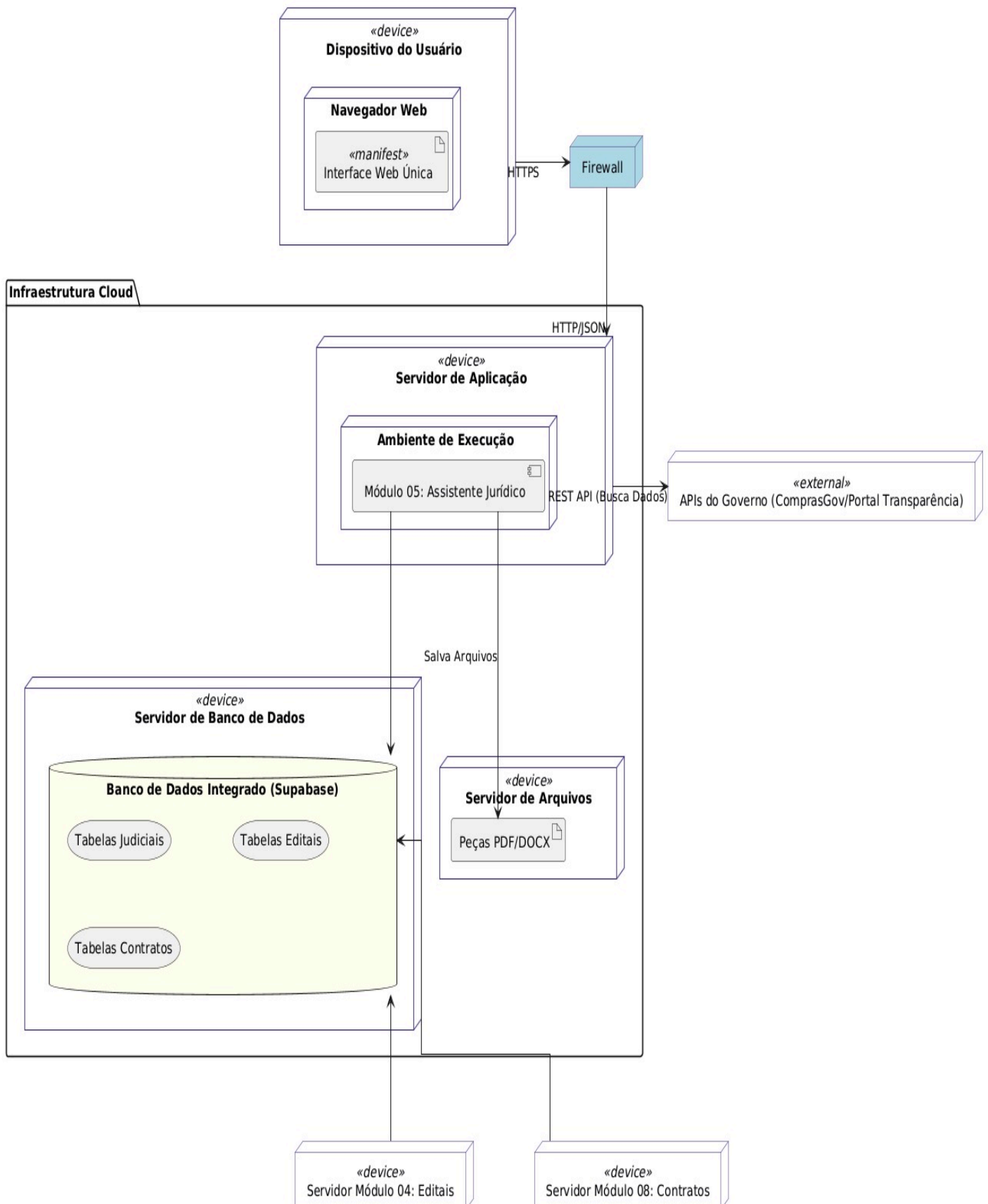
- Banco de Dados (PostgreSQL): Armazena de forma integrada as tabelas de teses, editais e as peças já geradas, permitindo a troca de informações com outros módulos de forma centralizada.
- Storage (Arquivos Digitais): Repositório dedicado para os documentos essenciais para o funcionamento do sistema, como editais e peças jurídicas.

→ Ecossistema Externo e Integrações: O Módulo 05 funciona de forma integrada com outros sistemas, conforme os requisitos do projeto:

- Módulo 04 (Editais) e Módulo 08 (Contratos): Fornecem os dados necessários para que a peça jurídica seja contextualizada com informações reais do pregão ou contrato.
- Governo APIs (ComprasGov): Fonte de dados externa para monitoramento de novos editais e prazos em tempo real.

2. Diagrama de Implantação

Diagrama de Implantação - Módulo 05 (Assistente Jurídico)



Detalhamento:

→ Camada de Cliente e Segurança:

- Acesso do Usuário: O acesso ocorre via Navegador Web em dispositivos PC ou Mobile. A comunicação é protegida pelo protocolo HTTPS, garantindo a segurança desde o ponto de origem.
- Proteção por Firewall: Antes de atingir a infraestrutura interna, todo o tráfego passa por um Firewall, que atua como uma barreira contra acessos não autorizados e filtra as requisições de API.

→ Servidor de Aplicação e Integração Governamental

- Ambiente de Execução (Módulo 05): O Servidor de Aplicação hospeda o Módulo 05 de forma centralizada. É neste ponto que reside a lógica de processamento e o motor de IA.
- Integração com Governo (REST API): Esta infraestrutura irá apresentar conexão direta com as APIs do Governo (ComprasGov / Portal da Transparência). O servidor realiza requisições REST para buscar dados atualizados sobre pregões e editais.

→ Persistência de Dados com Supabase

- Banco de Dados Integrado: O sistema utiliza o Supabase (PostgreSQL) como sua camada de dados principal.
- Unificação de Módulos: O banco centraliza as tabelas de módulos e frentes diversas, permitindo que o Módulo 05 tenha visão total do sistema.
- Conexões Diretas: As aplicações dos Módulos 04 e 08 conectam-se diretamente ao Banco Integrado, garantindo que o Assistente Jurídico sempre trabalhe com dados sincronizados.

→ Gestão de Arquivos e Armazenamento

- Servidor de Arquivos: Como o Módulo 05 é responsável pela geração automatizada de documentos, ele possui uma conexão direta para salvar as peças jurídicas em formato PDF/DOCX em um diretório independente. Isso evita sobrecarregar o banco de dados com arquivos pesados.

3. Architecture Decision Records (ADR) Documentado

Para fundamentar o projeto, definiu-se uma arquitetura considerada objetiva e ágil, priorizando a entrega rápida e a integração entre as frentes de trabalho. Como decidido em sala de aula, o Supabase centraliza os dados e garante que todos os módulos se comuniquem de forma fluida.

No campo das escolhas técnicas, estabeleceu-se maior foco na familiaridade dos membros com as ferramentas e na forte presença delas no ecossistema web. O objetivo central foi garantir uma curva de aprendizado menor e um desenvolvimento acelerado, fatores cruciais para o curto prazo do projeto. Em resumo, as tecnologias selecionadas foram:

- Banco de Dados e Integração: Supabase.
- Tecnologias de Aplicação: Next.js com React e Node.js.
- Versionamento e Hospedagem: GitHub e Vercel.

Para garantir a escalabilidade e a rastreabilidade da arquitetura definida para o projeto, foi criado um repositório no GitHub contendo uma pasta exclusiva para a organização dos ADRs: [Link do Repositório ADR](#).

4. Padrão Arquitetural Justificado

A arquitetura central adotada para este projeto é o padrão Cliente-Servidor (Client-Server), em uma abordagem que utiliza o conceito de Backend-as-a-Service (BaaS). Este padrão estabelece uma separação clara de responsabilidades entre duas camadas distintas que se comunicam por meio de requisições e respostas, geralmente sobre o protocolo HTTP:

- O Cliente: É responsável pela camada de apresentação e pela interação direta com o usuário.
- O Servidor: Centraliza o acesso aos dados, a lógica de negócio sensível e os serviços de infraestrutura e autenticação.

Nessa implementação específica, será utilizado o Next.js (hospedado na Vercel) como servidor da interface e das APIs intermediárias, enquanto o Supabase atua como o servidor de dados, autenticação e armazenamento de arquivos.

→ Justificativa Técnica e Comparativo

A escolha desta arquitetura em detrimento de outras alternativas populares se baseou principalmente nos seguintes critérios:

- Por exemplo, por que não Microsserviços?

Embora a arquitetura de microsserviços seja excelente para escalabilidade extrema e módulos isolados, ela introduz uma complexidade de infraestrutura (como DevOps, orquestração de contêineres e mensageria) que é incompatível com o prazo e a necessidade de um MVP (Produto Mínimo Viável) rápido. Microsserviços resolveriam problemas de escala que o projeto ainda não enfrenta.

- Por que não o Monolito Tradicional? (ex: Spring Boot ou Django)

Criar um backend monolítico do zero exigiria que a equipe programasse sistemas complexos de gerenciamento de sessões e armazenamento de arquivos de forma manual. Ao adotar o Supabase, é possível “terceirizar” essa complexidade técnica, ganhando tempo para focar nas funcionalidades principais do sistema e garantir a entrega de valor.

→ Encaixe com as Regras de Negócio (Assistente Jurídico)

O caso de uso principal do sistema exige a geração de documentos pesados (PDF/DOCX) e buscas complexas em teses jurídicas.

- Processamento Isolado: O padrão adotado permite que a geração pesada de arquivos ocorra de forma isolada no servidor (via Next.js API Routes), mantendo a interface do navegador leve e responsiva para o advogado.
- Busca Eficiente: O Supabase oferece busca textual (Full-Text Search) nativa no PostgreSQL, o que é ideal para o buscador de teses jurídicas, eliminando a necessidade de ferramentas externas adicionais.

→ Alinhamento com a Equipe (Fator Humano)

A escolha de utilizar JavaScript de ponta a ponta (Full Stack) centraliza as tecnologias, permitindo que todos colaborem no mesmo repositório sem a necessidade de alternar entre linguagens e tecnologias distintas. Esta divisão acelera o desenvolvimento ao remover a necessidade de configurar infraestruturas avançadas, mantendo uma organização clara entre as telas e os dados.

5. Atributos de Qualidade (RNFs)

A arquitetura do Assistente Jurídico (Módulo 5), baseada na combinação de Next.js, Vercel e Supabase, foi projetada para entregar uma solução simples e escalável. A seguir, destacam-se os requisitos não funcionais do sistema:

→ Desempenho (Velocidade e Processos):

A velocidade do sistema será garantida pela rede global de CDN da Vercel e pelas estratégias de renderização do Next.js, que entregam telas de carregamento rápidas e com baixa latência. Será implementado controle ágil no processamento de tarefas pesadas, como a geração de PDFs e DOCX, através de pequenos blocos de código escritos para executar uma tarefa única e específica na nuvem. Como essas funções possuem um limite de tempo de execução entre 10 a 60 segundos na conta gratuita da Vercel, a arquitetura e o código serão desenhados para otimizar a geração dos documentos estritamente dentro dessa janela, buscando evitar travamentos na interface do utilizador. Paralelamente, o recurso de Full-Text Search nativo do PostgreSQL viabiliza buscas complexas e quase instantâneas em grandes volumes de dados, como temas e teses jurídicas.

→ Manutenibilidade (Facilidade de Evolução):

A arquitetura prioriza a simplicidade de manutenção através de uma stack unificada em Node e React com o Next.js, estruturada para comportar o trabalho dos 6 membros da equipe. Isso elimina a complexidade de gerenciar repositórios separados para frontend e backend, facilitando a colaboração do time. Além disso, a forte separação modular entre interface, regras de negócio, APIs e banco de dados permite que o sistema evolua e receba novas integrações externas sem que alterações visuais quebrem outras partes do negócio, evitando o popular código espaguete.

→ Integridade dos Dados:

O uso do PostgreSQL garante o cumprimento das propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade). Em caso de falha no meio do processo, o sistema reverte a transação automaticamente, impedindo dados corrompidos ou parciais. Além disso, para manter o rigor jurídico, a base de dados de decisões administrativas será atualizada automaticamente em intervalos fixos de 72 horas, garantindo que a fundamentação legal seja sempre precisa e atual. O Supabase complementa essa camada com a parte de backups.

→ Disponibilidade e Resiliência (Tolerância a Falhas):

Por operar 100% em nuvem (Vercel e Supabase), o sistema funciona 24/7 com escalabilidade automática, suportando picos de acesso sem depender de servidores locais. A aplicação também foi desenhada para ser resiliente: caso serviços externos ou APIs governamentais fiquem fora do ar, as rotas do Next.js tratam as exceções e exibem mensagens amigáveis ao utilizador final, evitando o travamento completo da plataforma.

→ Segurança:

Sendo um sistema jurídico sujeito à LGPD, a segurança é tratada como prioridade base que envolve todo o sistema. O isolamento dos dados ocorre a nível de banco de dados. Na prática, isso garante um controle específico, onde cada fornecedor ou advogado tem acesso exclusivo apenas aos seus próprios tópicos, editais e peças jurídicas, protegidos por rotas autenticadas e criptografia HTTPS.